

Aula 1 — Hardware, CPU, RAM, armazenamento e processos

1. O que é um computador (visão simples)

Um computador é uma máquina que **recebe instruções, processa dados e produz um resultado**. Todo o resto — sistema operacional, aplicativos, antivírus, malware, SIEM — é, no fundo, um conjunto de instruções rodando sobre esse hardware.

As quatro peças que você precisa entender primeiro:

Peça	O que faz (simples)	Nome técnico
"Cérebro" que executa instruções	processa cada instrução, uma a uma (ou várias em paralelo)	CPU (Central Processing Unit)
"Mesa de trabalho" temporária, rápida, que some quando desliga	guarda o que está em uso agora	RAM (memória volátil)
"Arquivo/gaveta" permanente, mais lento, guarda mesmo desligado	guarda dados de forma permanente	Armazenamento (disco/SSD)
"Portas" de entrada/saída de dados	conectam o computador a periféricos e à rede	Periféricos e interfaces de rede

2. CPU

A CPU executa instruções, uma atrás da outra (ou várias em paralelo, se tiver múltiplos núcleos/cores). Todo programa — do Chrome a um ransomware — é, no fim, uma sequência de instruções que a CPU executa.

Por que importa para segurança: uso anômalo de CPU pode indicar mineração de criptomoeda não autorizada, um processo em loop malicioso, ou ransomware criptografando arquivos em massa. É um dos primeiros sinais que um analista observa em um host "lento sem motivo aparente".

3. RAM (memória volátil)

A RAM guarda os dados que estão **em uso agora**: programas abertos, variáveis, o conteúdo que a CPU está processando no momento. Quando o computador desliga, o conteúdo da RAM **desaparece**.

Por que importa para segurança: muito malware moderno roda **apenas na memória** ("fileless malware") para não deixar arquivo no disco e escapar de antivírus tradicionais. Isso é o motivo pelo qual existe uma especialidade chamada **memory forensics** (análise forense de memória) — sem ela, esse tipo de ataque pode passar despercebido, porque some ao desligar a máquina. Vamos estudar isso a fundo no módulo de Digital Forensics.

4. Armazenamento (disco/SSD)

Guarda dados de forma **permanente**: o sistema operacional, programas instalados, arquivos do usuário, logs. Sobrevive a um desligamento.

Por que importa para segurança: é onde ficam os **artefatos** que um investigador forense analisa depois de um incidente — arquivos criados, modificados, apagados (mas muitas vezes recuperáveis), timestamps, logs salvos em disco. É também onde ransomware criptografa os arquivos da vítima.

5. Processos

Um **processo** é um programa em execução. Quando você abre o navegador, o sistema operacional cria um processo para ele: reserva um espaço de RAM, associa um identificador único (**PID** — Process ID) e começa a executar suas instruções na CPU.

Conceitos essenciais:

- **Processo pai e processo filho:** um processo pode iniciar outro. Ex.: o Explorer (processo pai) abre o Word (processo filho) quando você clica duas vezes num arquivo `.docx`. Essa relação pai/filho é registrada pelo sistema operacional e é **uma das evidências mais importantes** em investigação de segurança.
- **Command line (linha de comando):** os argumentos com que um processo foi iniciado. Ex.: `powershell.exe -enc <base64>` é muito diferente de só `powershell.exe` — o primeiro é um forte indicador de comando ofuscado, comum em ataques.
- **Usuário do processo:** todo processo roda "como" algum usuário (ex.: `SYSTEM`, `root`, ou o usuário logado). Um processo crítico do sistema sendo executado por um usuário comum, ou vice-versa, é suspeito.

Por que importa para segurança: a maior parte da detecção em endpoint (antivírus, EDR) depende de observar a **árvore de processos**: quem iniciou quem, com quais argumentos, e o que esse processo fez em seguida (abriu conexão de rede? criou um arquivo? modificou o Registry do Windows?). Exemplo clássico e muito estudado em Blue Team: `winword.exe` (Word) criando como processo filho um `powershell.exe` — Word não deveria, em uso normal, abrir PowerShell. Isso é um padrão típico de um documento malicioso com macro.

6. Conectando os pontos

```
CPU processa → RAM guarda o que está em uso → Disco guarda o permanente
      ↓
    Processos
  (programas em execução, com PID,
 processo pai/filho, usuário, command line)
      ↓
Comportamento anômalo de processo = pista de segurança
```

Isso é a base de tudo que vem depois: quando estudarmos Sysmon, EDR e Windows Event Logs, você vai ver esses mesmos conceitos (PID, processo pai/filho, command line, usuário) aparecendo repetidamente como os campos centrais de qualquer investigação.

7. Exercício prático (sem instalar nada)

Se você tiver acesso a um terminal Linux ou macOS agora, rode:

```
ps -ef | head -20
```

E tente identificar, em pelo menos 3 linhas do resultado:

1. O **PID** do processo.
2. O **usuário** que está executando (segunda ou terceira coluna, dependendo do sistema).
3. O **comando** que foi executado (última coluna) — é isso que corresponde à "command line" que explicamos acima.

Não se preocupe em entender a saída inteira agora — o comando `ps` e o resto do terminal Linux serão ensinados em detalhe no **Módulo 01 — Linux para Blue Team**. O objetivo aqui é só começar a enxergar "processo" como uma coisa concreta na tela, não apenas um conceito abstrato.

Se não tiver terminal disponível agora, sem problema — retomamos isso com calma no próximo módulo.

8. Recapitulando

- Computador = CPU (processa) + RAM (memória temporária) + Disco (permanente).
- Processo = um programa rodando, identificado por PID, com processo pai/filho, usuário e command line.
- Esses quatro campos (PID, pai/filho, usuário, command line) são a base de praticamente toda detecção baseada em processos que você vai estudar daqui para frente.

Próxima aula

Sistemas operacionais, usuários, grupos e permissões — como o sistema operacional decide **quem pode fazer o quê**, e por que isso é a base do princípio de menor privilégio em segurança.

Fontes recomendadas

- **Microsoft Learn — "Processes and Threads"**: explica oficialmente como o Windows gerencia processos e threads. <https://learn.microsoft.com/windows/win32/procthread/processes-and-threads>
- **NIST SP 800-61 Rev. 2 (Computer Security Incident Handling Guide)**: referência oficial sobre como artefatos de processo/memória entram em uma investigação de incidente. <https://csrc.nist.gov/pubs/sp/800/61/r2/final>